

不利于提高物理资源的利用率，虚拟机一般无法使用完物理设备的所有资源，例如虚拟机很难每时每刻都消耗掉一张网卡的所有带宽。我们该如何解决这些问题？答案就是使用 SR-IOV（Single Root I/O Virtualization）技术。

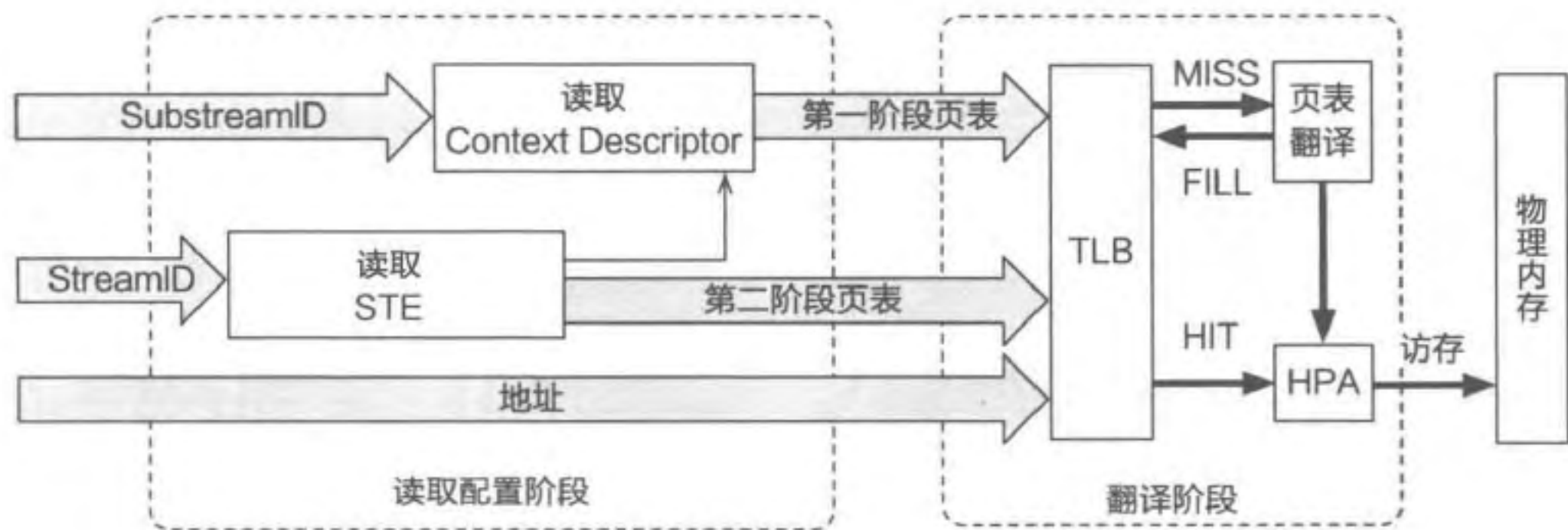


图 14.24 SMMU 中的地址翻译过程^[5]

SR-IOV 是一个标准化规范，满足 SR-IOV 规范的设备能够在硬件层面实现 I/O 虚拟化。具体来说，该物理设备可创建一个“真身”和多个“分身”，“真身”称为 Physical Function（PF），“分身”称为 Virtual Function（VF）。虚拟机监控器使用 PF 创建和管理 VF，并将每个 VF 直通给一台虚拟机管理。虚拟机通过 DMA、MMIO 等方式与 VF 直接交互，在数据传输的过程中不会下陷到虚拟机监控器中。设备在硬件层面实现 VF 之间的隔离，虚拟机监控器也需要使用 IOMMU 限制每个 VF 能访问的主机物理地址。

在使用 SR-IOV 和 IOMMU 之后，每台虚拟机可独占式地使用直通设备，在数据传输过程中避免虚拟机下陷，从而提升 I/O 虚拟化的性能。然而，由于 I/O 过程完全绕开虚拟机监控器，虚拟机监控器无法在任意时刻检查虚拟机的状态，因而难以支持虚拟机热迁移以及虚拟机安全自省等功能。

14.5.4 小结

表 14.5 总结了本节介绍的三种 I/O 虚拟化方法，基于软件的技术包括软件模拟与半虚拟化，基于硬件虚拟化的技术为设备直通。图 14.25 进一步展示了四种 I/O 虚拟化方法的架构对比（其中半虚拟化方法分成两种子类型）。软件模拟方法（图 14.25a）作用于硬件指令层，因而可以透明地提供对虚拟 I/O 设备的支持。由于大量的虚拟机下陷以及无法批量传输大量 I/O 数据，该方法的性能并不理想。半虚拟化方法通过主动降低透明性来换取性能的提升（图 14.25b），也可以通过将后端驱动直接运行在内核态来进一步提升性能（图 14.25c）。与基于软件的方法不同，设备直通可以安全而高效地将某个物理设备以 VF 的形式直通

表 14.5 I/O 虚拟化技术总结

软件技术	硬件虚拟化技术
软件模拟方法	设备直通（IOMMU 和 SR-IOV）
半虚拟化方法	

给一台虚拟机使用，从而进一步提升 I/O 虚拟化的性能（图 14.25d）。

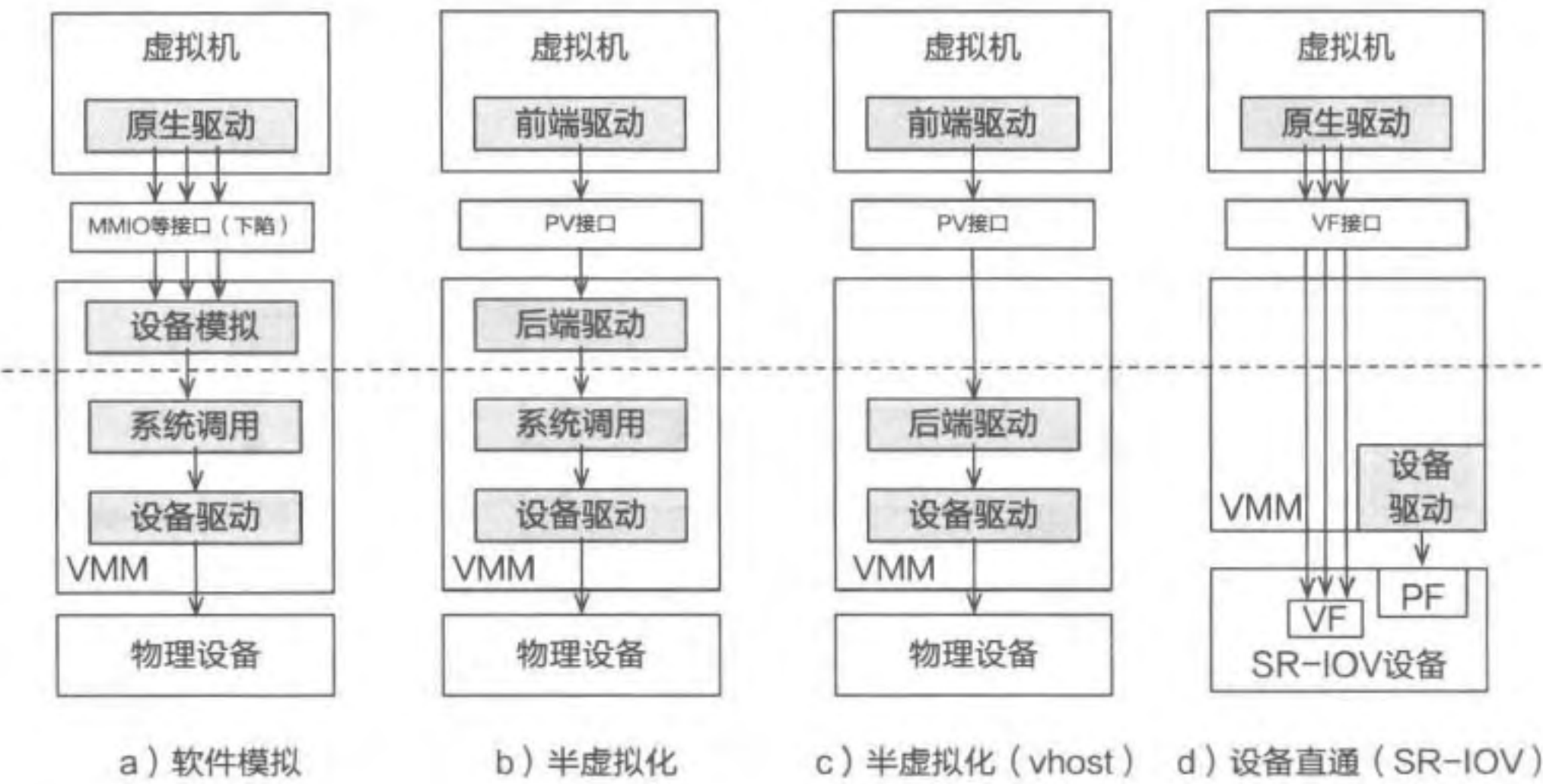


图 14.25 I/O 虚拟化方法的架构对比

14.6 中断虚拟化

本节主要知识点

□ 如何在 AArch64 架构中实现中断虚拟化？

中断是操作系统与设备交互时必不可少的机制。在完成某任务后，设备可通过中断异步地通知 CPU，以便其进行后续操作。例如，当网卡设备接收到网络包时，它可发送中断异步地通知 CPU，CPU 随后调用操作系统注册在 VBAR_ELx 寄存器中的中断处理函数。

在非虚拟化场景中，来自设备的中断直接唤醒操作系统配置的中断处理函数。虚拟化场景中的中断传输情况稍显复杂，分为两种类型，一种是物理中断，一种是虚拟中断。物理中断由物理设备直接产生，交给虚拟机监控器处理。虚拟机监控器一般不允许设备直接将物理中断发送给虚拟机，这主要是因为物理设备的中断不一定和某个虚拟机直接相关，例如，有可能此时网卡收到的网络包属于另一个没有运行的虚拟机。虚拟中断不由物理设备直接产生，而由虚拟机监控器（或下文介绍的 ITS）产生并传递给虚拟机。

在推出硬件虚拟化技术之前，中断虚拟化需通过软件模拟的方式实现。一方面，软件模拟方法中的所有中断寄存器本质上都是内存中的状态，这些虚拟寄存器的状态改变需要由虚拟机监控器维护。对于虚拟机而言，寄存器所在的内存必须被设置为不可访问。

因此,如果虚拟机需要访问相关寄存器(例如读取 ICC_IAR1_EL1 寄存器以获得当前中断号),将触发异常并进入虚拟机监控器,虚拟机监控器之后模拟寄存器的访问过程(如将读取结果写入目标寄存器)。另一方面,在插入虚拟中断时,虚拟机监控器直接调用虚拟机内的中断处理函数。之后,虚拟机读取 ICC_IAR1_EL1 寄存器获取中断号,并处理中断。通过软件模拟的方法实现中断会带来较多的虚拟机下陷,因而性能较差。

为了优化中断虚拟化的性能,ARM GIC (Generic Interrupt Controller) 在第二版中增加了对中断虚拟化的硬件支持。自此之后,虚拟机监控器可以为虚拟机配置硬件实现的虚拟寄存器,虚拟机可使用这些寄存器控制中断的行为(例如开关中断)。虚拟机在访问这些寄存器时不会产生任何虚拟机下陷。此外,插入中断的方式也得到了优化。虚拟机监控器可以使用 GIC 提供的接口插入中断,此中断将在 CPU 恢复虚拟机执行时由硬件自动插入并直接调用对应的中断处理函数。

硬件虚拟化中的虚拟中断插入过程

当物理设备产生物理中断并发送到某一个物理核时,如果此时虚拟机正在运行,将会引起该虚拟机下陷。在处理完此物理中断后,虚拟机监控器判断出该物理中断相关的 I/O 数据需要传输给某虚拟机。因此,它先将数据拷贝进虚拟机内存区域,并向此虚拟机插入虚拟中断。为了插入虚拟中断,虚拟机监控器在 GICH_LR<n>_EL2 寄存器中写入该虚拟中断的信息,例如中断号和优先级等。之后,虚拟机监控器恢复 vCPU 的执行,在这一时刻,硬件自动根据 GICH_LR<n>_EL2 中的信息向 vCPU 插入一个中断,并调用其中断处理函数。在虚拟机处理该中断的过程中,写入 GICC_EOIR1_EL1 等寄存器时不会触发任何虚拟机下陷。

上述中断虚拟化方法虽然已经大大提升了虚拟机在处理中断时的性能,但每次触发物理中断时依然会引起虚拟机下陷。尤其是当设备已经直通给虚拟机时,由物理中断产生的下陷会对虚拟机的 I/O 性能产生较大的影响。为了解决该问题,不同体系结构推出了新的中断虚拟化技术,允许物理设备直接向虚拟机发送物理中断(不产生虚拟机下陷)。这种技术在 Intel 体系结构下被称为 Posted Interrupt,它的转发机制主要由 IOMMU 实现。在 AArch64 体系结构下,这种中断由 GIC 的 ITS (Interrupt Translation Service) 实现。

在调度虚拟机之前,虚拟机监控器配置 ITS 中的中断翻译表 (Interrupt Translation Table, ITT) 等信息,将直通设备的物理中断映射成某个虚拟中断。直通设备发送物理中断后, GIC 查询中断翻译表等信息,并将物理中断的信息翻译成对应的虚拟中断。之后, GIC 直接将该虚拟中断插入对应的虚拟机,并调用客户操作系统注册的中断处理函数。处理此中断的过程与非虚拟化场景下完全一致,不会产生任何虚拟机下陷。因此,使用该技术之后,物理中断的产生、翻译、插入和处理过程不需要虚拟机监控器的介入,完全由设备和 GIC ITS 配合完成,可进一步提升中断虚拟化的性能。

14.7 案例分析：QEMU/KVM

本节主要知识点

- ❑ KVM 的接口是什么？如何在用户态使用这些接口？
- ❑ KVM 和 QEMU 如何协同工作？
- ❑ KVM 的运行机制是什么？

KVM (Kernel-based Virtual Machine) 是运行在 Linux 内核中的硬件虚拟化管理模块，用户态的程序可通过 KVM 暴露的用户态接口使用硬件虚拟化的功能。图 14.26 展示了 KVM 与用户态程序以及虚拟机的关系。首先，在用户态程序运行之前，KVM 通过 Linux 在 /dev 目录下注册并初始化 kvm 设备文件。之后，运行在 EL0 的用户态程序可通过 `ioctl` 等系统调用与 /dev/kvm 设备文件交互，以调用内核态 KVM 的 API。KVM 收到 API 调用后，在 Linux 内核中创建相应的虚拟机、vCPU、第二阶段页表等数据结构。如图 14.26 所示，此用户态程序调用了 KVM 的 `KVM_SET_USER_MEMORY_REGION` API 后，KVM 据此建立该虚拟机的第二阶段页表。在此程序调用 `KVM_RUN` API 时，KVM 使用 `eret` 指令进入某 vCPU 相关的代码位置开始执行。

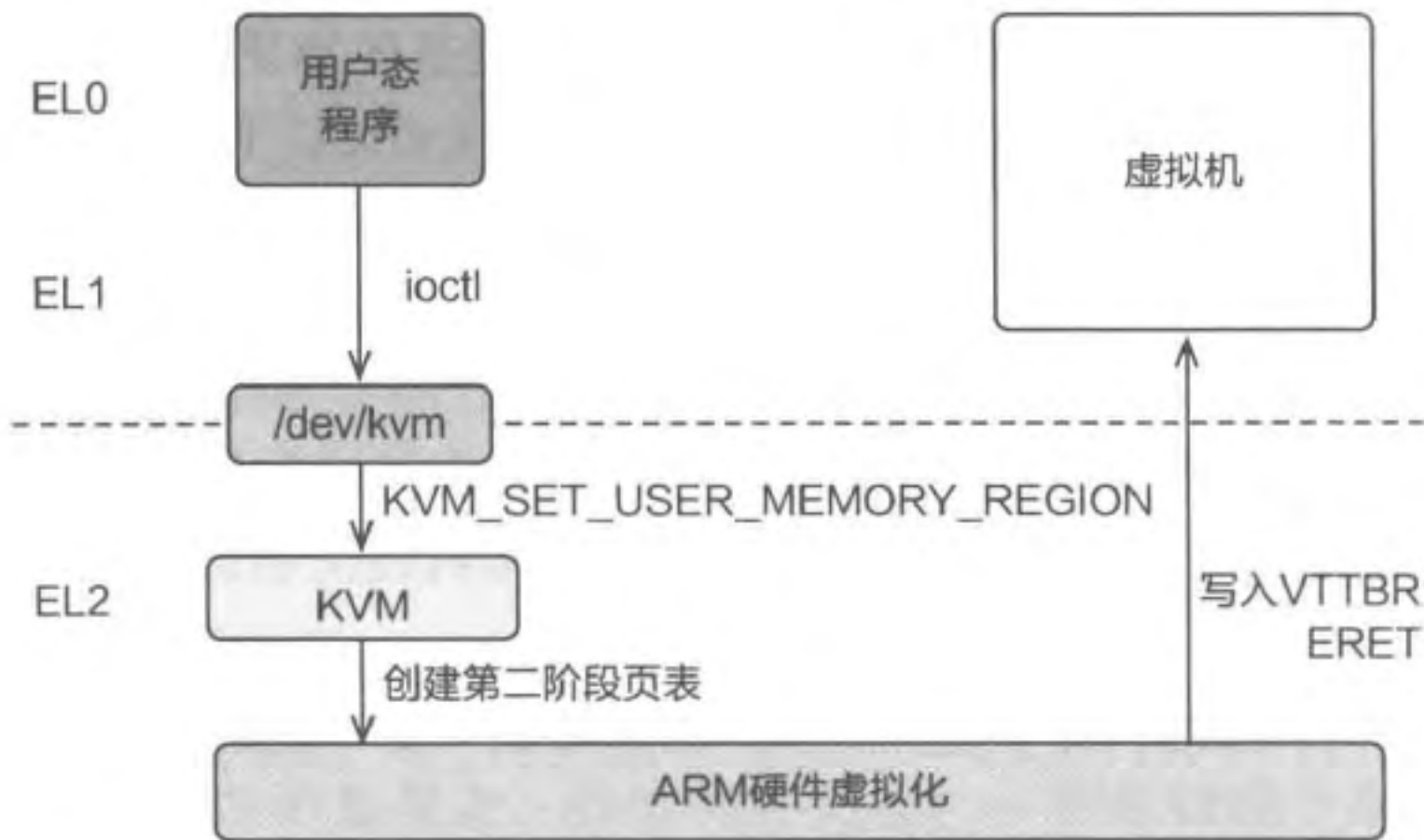


图 14.26 用户态程序使用 KVM 接口运行虚拟机

14.7.1 KVM API 和一个简单的虚拟机监控器

KVM 向用户态提供了 100 多个不同的 API，以使用户态的程序对虚拟机进行管理 (表 14.6 展示了其中 7 个重要的 API)。通过这些 API，用户态程序可以完成创建虚拟机、创建虚拟机内的 vCPU、设置 vCPU 的寄存器状态等操作。读者可能会认为写一个虚拟机监控器非常复杂，实际上，通过 KVM 的 API，使用短短几十行代码，我们就可以创建一个非常简单的虚拟机监控器。